Project - High Level Design

On

RAG-Retail-Chat

Course Name: GEN-AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Aryan Gangrade	EN22IT301023
2.	Suzal Hasam Ali Devani	EN22IT301111
3.	Ayushi Jat	EN22IT301028
4.	Naina Sahu	EN24CA5030106
5.	Aditi Singh	EN24CA5030008
6.	Preeti Prajapat	EN24CA5030131

Group Name: Group11 D12

Project Number: GAI-21

Industry Mentor Name: Mr.Yash Bansal

University Mentor Name: Prof. Ajaj khan

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

1.1 Scope of the Document :-

This High Level Design (HLD) document provides a comprehensive overview of the RAG Retail Chat application, which is an AI-powered chatbot system designed for retail query handling. The purpose of this document is to describe the overall architecture, system design, data flow, components, and integration points at a high level.

The document focuses on major functional components such as query processing, retrieval mechanisms, and response generation using Large Language Models (LLMs). It also covers non-functional aspects such as performance, scalability, and security.

This document does not include low-level implementation details such as source code or algorithm-level design. It is intended to give a clear understanding of how the system is structured and how different components interact.

1.2 Intended Audience :-

The intended audience for this document includes faculty members, project evaluators, system architects, developers, and stakeholders involved in understanding or reviewing the system design.

This document helps readers understand the architecture, working flow, and technologies used in the system without going into deep coding details.

1.3 System Overview :-

RAG Retail Chat is an intelligent chatbot system built using the Retrieval-Augmented Generation (RAG) architecture. The system is designed to assist users by answering retail-related queries such as product details, availability, and recommendations.

The system works by combining two major techniques:

- Information Retrieval (from stored retail data)
- Text Generation using LLM

When a user submits a query, the system retrieves relevant data from a vector database and uses a language model to generate a context-aware response.

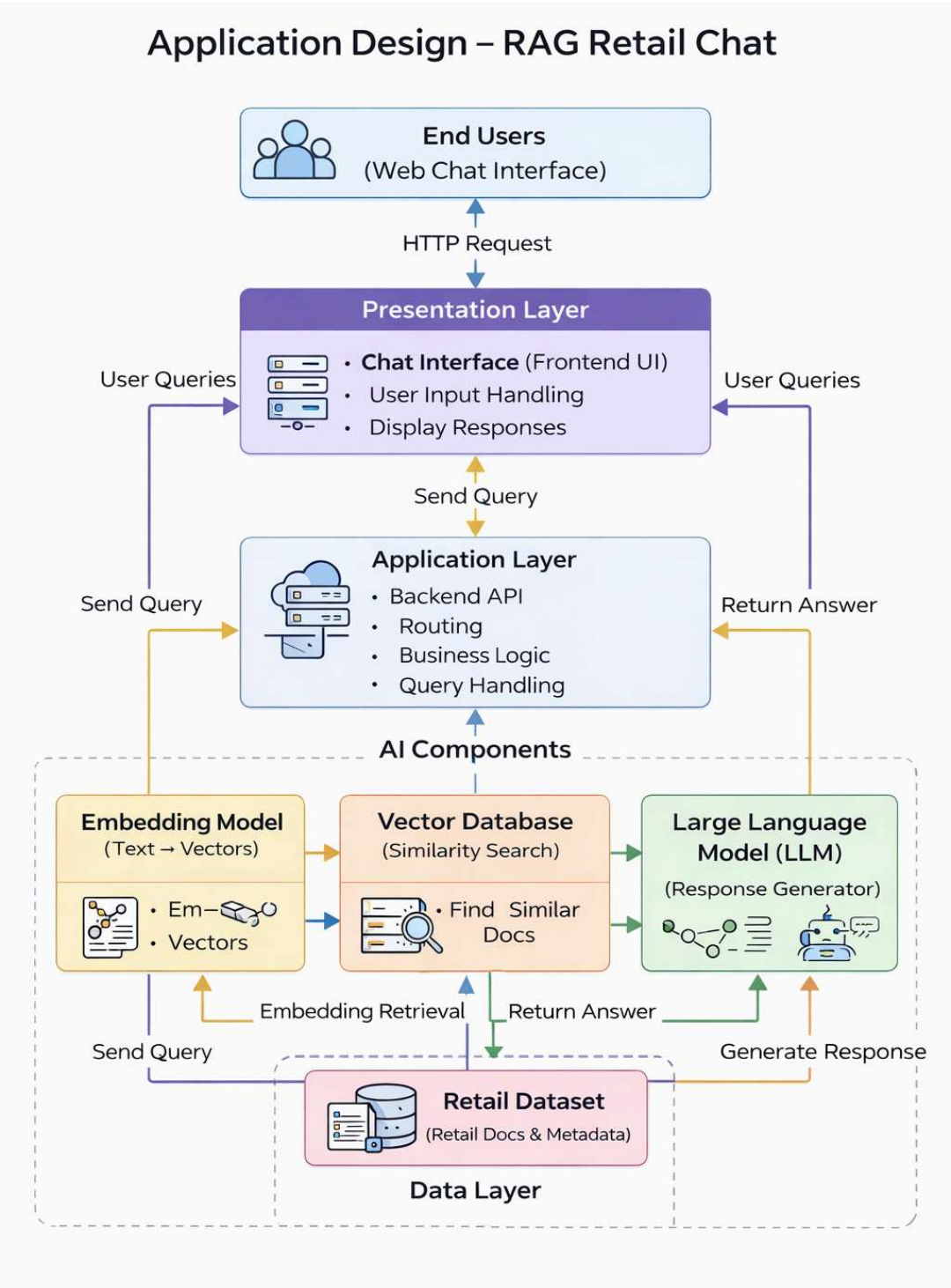
The application follows a client-server architecture where:

- The frontend provides a chat interface
- The backend processes queries
- The AI layer handles embeddings and response generation

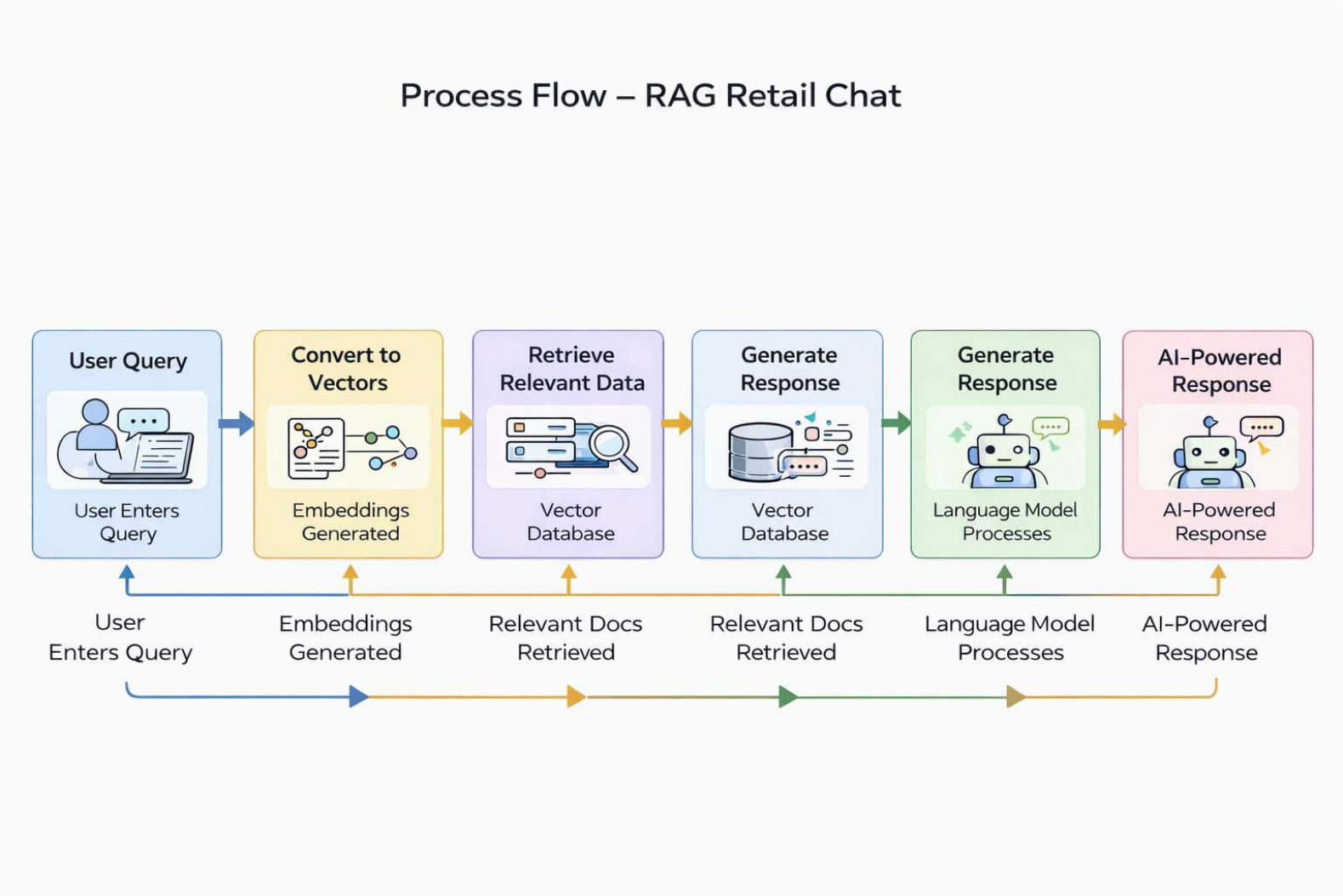
.

2. System Design

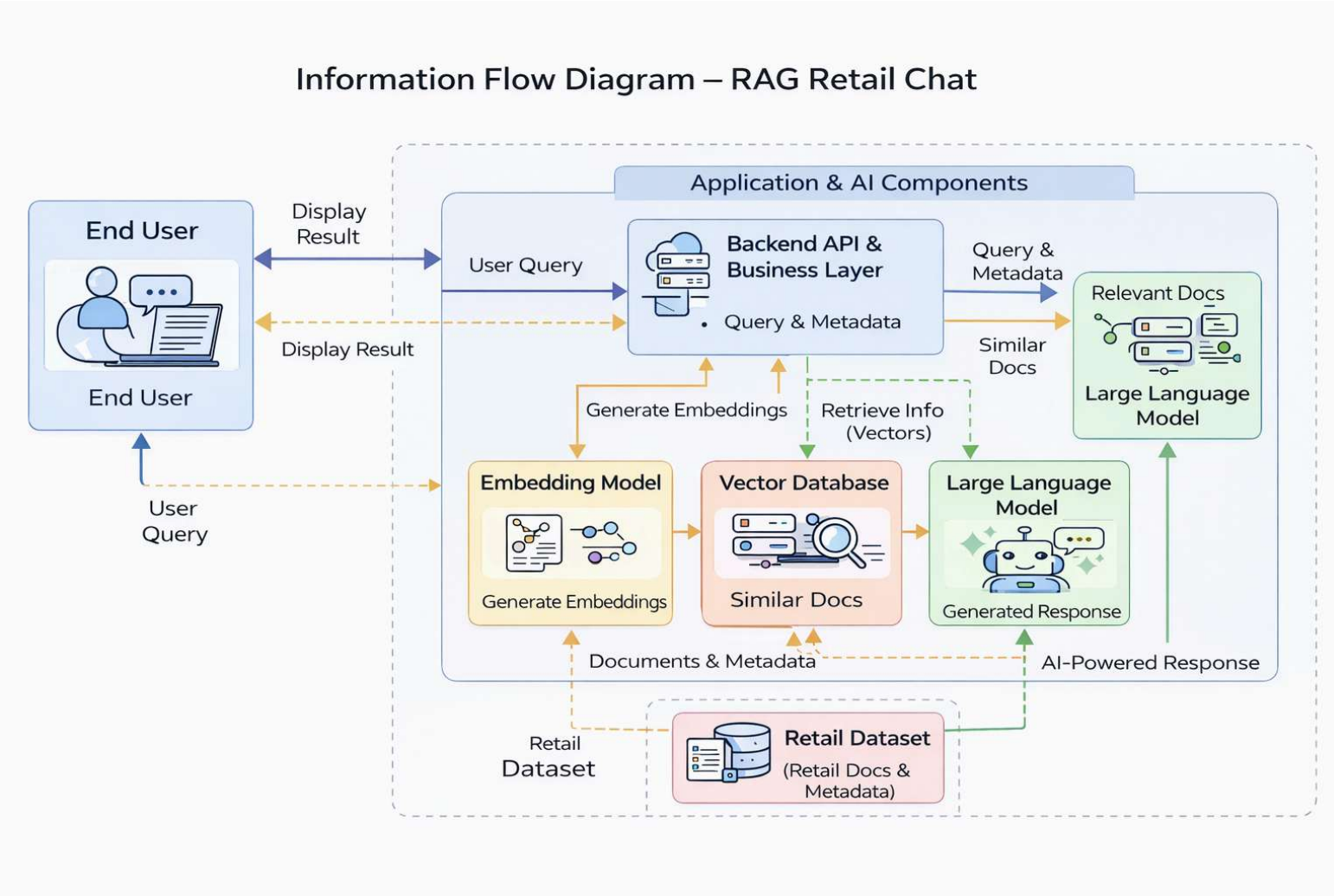
2.1 Application Design



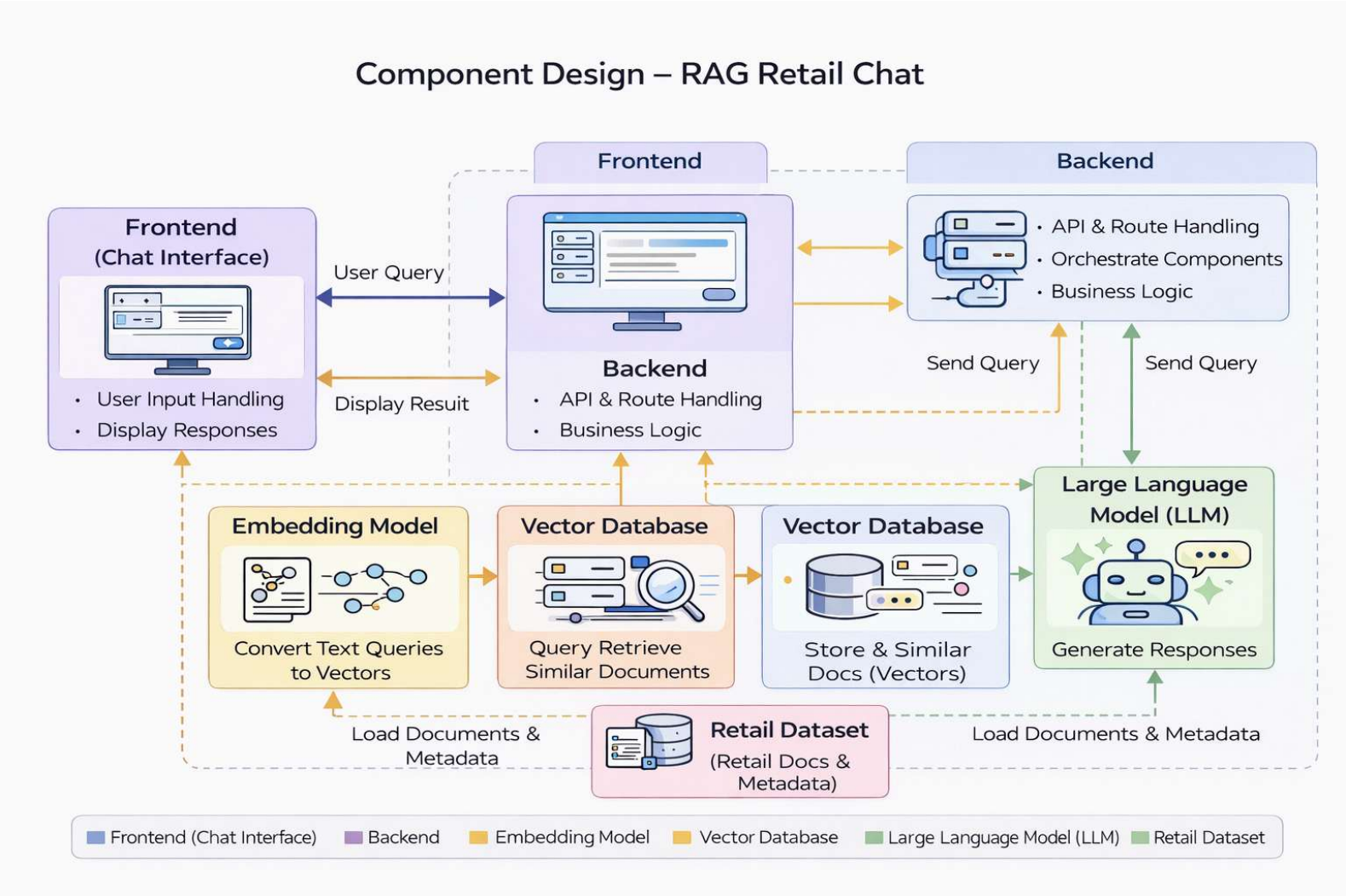
2.2 Process Flow



2.3 Information Flow



2.4 Component Design



2.5 Key Design Considerations

1. Simplicity

The system is designed with simplicity in mind to ensure ease of development, understanding, and maintenance.

- The architecture follows a modular layered approach, separating frontend, backend, AI components, and data layers.
- Each component has a clearly defined responsibility, reducing complexity.
- Minimal dependencies are used to avoid unnecessary overhead.
- The system is easy to understand for developers and suitable for academic as well as real-world use cases.

2. Accuracy (RAG-based Design)

Accuracy is a key factor in chatbot systems, and this project uses Retrieval-Augmented Generation (RAG) to improve response quality.

- Instead of generating answers purely from the language model, the system retrieves relevant data first.
- The retrieved context is then passed to the LLM to generate more accurate and factual responses.
- Reduces hallucination (incorrect answers) commonly seen in traditional AI chatbots.
- Ensures that responses are context-aware and domain-specific (retail-focused).

3. Scalability

The system is designed to handle future growth in terms of users, data, and features.

- The architecture supports easy integration with cloud platforms.
- Vector databases can scale to handle large volumes of embeddings and documents.
- The backend can be extended into microservices architecture if required.

- New features such as recommendation systems, analytics, or multi-language support can be added easily.

4. Performance Optimization

Performance is critical to provide fast and smooth user experience.

- Use of vector databases (like FAISS/Pinecone) ensures fast similarity search.
- Embeddings reduce computation time during query processing.
- Efficient API handling reduces response latency.
- Optional caching mechanisms can be added for frequently asked queries.

5. Maintainability

Maintainability ensures that the system can be easily updated and debugged.

- Clear separation between UI, backend logic, and AI components.
- Modular code structure allows independent updates of each component.
- Use of standard frameworks and libraries simplifies development.
- Easy integration of new datasets or models without affecting existing functionality.

6. Data Integrity

The system ensures that retrieved and processed data remains accurate and consistent.

- Retail data is stored in structured formats.
- Each document is converted into embeddings for reliable retrieval.
- Data consistency is maintained between dataset and vector database.
- Prevents incorrect or mismatched responses.

7. Reliability

The system is designed to provide consistent and dependable performance.

- Stable backend API ensures uninterrupted service.
- Error handling mechanisms manage failures gracefully.

- AI components are designed to handle incomplete or ambiguous queries.
- System can recover from failures without major disruption

2.6 API Catalogue

The Task Atlas application exposes the following internal APIs (routes) to support its functionality:

API Endpoint	HTTP Method	Description
/chat	POST	Accepts user query and returns AI-generated response
/embed	POST	Converts user query into vector embeddings
/retrieve	GET / POST	Retrieves relevant documents from vector database
/generate	POST	Generates final response using LLM
/health	GET	Checks system health/status

3. Data Design

The RAG Retail Chat system relies heavily on efficient data handling because its performance and accuracy depend on how well it retrieves and processes information. The data design combines structured retail data with vector-based representations (embeddings) to enable semantic search and intelligent response generation.

The system is designed to ensure:

- Fast retrieval of relevant information
- Accurate mapping between queries and documents
- Scalability for large datasets
- Data consistency and integrity

The data design is divided into four major parts: Data Model, Data Access Mechanism, Data Retention Policies, and Data Migration.

3.1 Data Model

The data model of the system defines how different types of data are structured and how they relate to each other within the application. In the RAG Retail Chat system, the data model consists of multiple logical entities that work together to support query processing and response generation.

- The first entity is the Query entity, which represents the input provided by the user through the chat interface. Every time a user submits a query, it is treated as a distinct record in the system. This entity typically contains attributes such as a unique query identifier, the text of the query, and a timestamp indicating when the query was made. In cases where user authentication is implemented, a user identifier may also be included. The primary purpose of this entity is to track user interactions and maintain a reference for generating and storing responses.
- The second entity is the Document entity, which serves as the core knowledge base of the system. This entity contains retail-related data such as product names, descriptions, categories, and other relevant metadata. Each document represents a piece of information that can be retrieved during query processing. The structure of this entity is designed to be flexible so that additional attributes like price, brand, or ratings can be included as needed. This entity is essential because it provides the contextual information that the system uses to generate meaningful responses.
- The third entity is the Embedding entity, which is fundamental to the functioning of the RAG architecture. In this system, both user queries and documents are converted into numerical vectors known as embeddings. These embeddings capture the semantic meaning of the text and allow the system to perform similarity-based searches. Each embedding is associated with a specific document through a reference key, ensuring that retrieved vectors can be mapped back to their original

content. This entity enables efficient semantic retrieval and significantly improves the accuracy of the system.

- The fourth entity is the Response entity, which stores the output generated by the system. Although storing responses is optional, it is useful for analysis, debugging, and system improvement. This entity typically includes a response identifier, the associated query identifier, the generated response text, and a timestamp. By maintaining this data, the system can evaluate performance and identify areas for enhancement.
- The relationships between these entities are designed to ensure smooth data flow. Each query is associated with a response, while each document is linked to an embedding. Additionally, a single query may retrieve multiple documents during the retrieval phase. This relational structure ensures that the system can efficiently map user input to relevant data and generate appropriate responses.

3.2 Data Access Mechanism

The data access mechanism of the RAG Retail Chat system is designed to enable efficient retrieval and processing of information. Unlike traditional systems that rely on keyword-based searches, this system uses a semantic retrieval approach powered by vector embeddings.

When a user submits a query, the system first converts the text into a vector representation using an embedding model. This vector is then compared with precomputed embeddings stored in the vector database. The database performs a similarity search to identify documents that are most relevant to the query. This process ensures that the retrieved information is contextually accurate rather than just matching keywords.

The vector database plays a central role in this mechanism. It is optimized for handling high-dimensional data and supports operations such as nearest neighbor search. Technologies like FAISS or Pinecone are commonly used for this purpose, as they provide fast and scalable retrieval capabilities. Once the relevant documents are retrieved, they are passed to the language model for response generation.

The backend API layer facilitates communication between the frontend interface and the data layer. It handles requests, processes queries, and coordinates interactions with the embedding model and vector database. This ensures that the entire retrieval and generation process is seamless and efficient.

In addition to retrieval, the system also supports data storage operations. When new documents are added to the system, they are converted into embeddings and stored in the vector database. This allows the system to continuously expand its knowledge base without affecting existing functionality. Overall, the data access mechanism ensures high performance, scalability, and accuracy in information retrieval.

3.3 Data Retention Policies

The data retention policies of the RAG Retail Chat system are designed to ensure efficient storage management while maintaining user privacy and system performance. User queries are stored temporarily for logging and analytical purposes, which helps in improving system accuracy and understanding user behavior, but they are not retained permanently and can be deleted periodically. Generated responses may also be stored optionally for evaluation and system improvement. The retail dataset, which acts as the core knowledge base, is stored permanently and updated regularly to ensure accurate responses. Additionally, embeddings are persistently stored in the vector database to avoid repeated computation and enhance retrieval speed. The system avoids storing sensitive user data and ensures secure handling through proper validation and controlled API access.

.

3.4 Data Migration

Data migration in the RAG Retail Chat system refers to the process of loading, updating, and managing data as the system evolves. Initially, the retail dataset is loaded into the system and converted into embeddings, which are stored in the vector database to enable semantic search. As new data is introduced, it is added incrementally without reprocessing the entire dataset, ensuring efficiency and scalability. The system is designed with a flexible schema that allows easy addition of new attributes such as ratings or product details without affecting existing data. It also supports migration from local storage to cloud-based infrastructure for better scalability and performance. Backup and recovery mechanisms can be implemented to ensure data reliability and minimize the risk of data loss.

4. Interfaces

The RAG Retail Chat system consists of multiple interfaces that enable smooth communication between different components of the application, including the user, backend services, and external systems. These interfaces ensure seamless interaction, data exchange, and integration across the entire system. The primary interface is the User Interface (UI), which allows users to interact with the system through a chat-based web application. Users can enter queries related to retail products, and the interface displays responses generated by the system in a clear and user-friendly manner. The UI is responsible for capturing user input, sending it to the backend, and presenting the output effectively, ensuring a smooth user experience.

The system also includes Application Programming Interfaces (APIs) that act as a communication bridge between the frontend and backend components. These APIs handle tasks such as receiving user queries, processing requests, retrieving relevant data from the vector database, and generating responses using the language model. The APIs follow a RESTful architecture, making them easy to integrate, scalable, and reusable. Additionally, the system interacts with external interfaces, such as language model APIs and vector database services, which provide advanced capabilities like embedding generation and semantic search.

4.1 User Interface (UI)

- Provides a chat-based interface for user interaction
- Allows users to enter retail-related queries
- Displays AI-generated responses in a readable format
- Ensures smooth and user-friendly experience
- Handles input and output communication with backend

4.2 Application Programming Interface (API)

- Acts as a bridge between frontend and backend
- Handles request and response processing
- Processes user queries and sends them to AI modules
- Retrieves data from vector database
- Generates responses using LLM
- Follows RESTful architecture for scalability and flexibility

4.3 External Interfaces

- Integrates with LLM APIs (e.g., OpenAI) for response generation
- Connects with vector databases (e.g., FAISS, Pinecone) for retrieval
- Supports embedding models for text-to-vector conversion
- Enables advanced AI capabilities and semantic search

4.4 System Interaction Interface

- Manages communication between internal components
- Ensures smooth data flow across layers
- Coordinates between backend, AI modules, and database
- Maintains system consistency and efficiency

5. State and Session Management

The RAG Retail Chat system follows a stateless architecture in which each user request is processed independently without storing permanent session data on the server, ensuring better scalability and performance. However, to maintain continuity in conversations, a unique session ID can be generated for each user interaction, allowing temporary tracking of requests within a session. The system may handle chat context by passing previous interactions along with the current query, enabling the generation of more accurate and context-aware responses. This context can be stored temporarily in memory or cache and does not require long-term persistence. Optional session storage mechanisms can be implemented if features like chat history or personalization are needed. Additionally, the stateless design allows the system to handle multiple users simultaneously without server dependency. From a security perspective, session IDs are managed securely, and no sensitive user data is stored within sessions, ensuring safe and reliable system operation.

6.Caching

Caching in the RAG Retail Chat system is implemented to improve performance, reduce response time, and optimize resource utilization. Frequently asked queries and their corresponding responses can be stored in a cache so that repeated requests can be served quickly without reprocessing the entire pipeline. Additionally, embeddings generated for common queries or documents can be cached to avoid redundant computation, which significantly reduces processing overhead. The system may use in-memory caching mechanisms such as Redis or local memory to store temporary data for fast access. Caching also helps in reducing the number of calls made to external services like language models and vector databases, thereby improving efficiency and lowering operational costs. The cache is typically short-lived and can be updated or invalidated periodically to ensure that users receive the most accurate and up-to-date information. Overall, caching enhances system responsiveness, scalability, and efficiency.

7. Non-Functional Requirements

The non-functional requirements of the RAG Retail Chat system define the quality attributes that ensure the system operates efficiently, securely, and reliably under different conditions. These requirements focus on aspects such as performance, scalability, security, and usability, which are essential for delivering a smooth and effective user experience. The system is designed to handle real-time interactions, maintain consistency in responses, and support future growth without compromising performance.

7.1 Security Aspects

- **Secure Communication:** All data exchange between frontend and backend is conducted over HTTPS, ensuring encrypted communication and protection from unauthorized access.
- **Input Validation:** The system validates all user inputs to prevent malicious activities such as SQL injection, prompt injection, or invalid data processing.
- **Data Privacy:** Sensitive user information is not stored permanently, ensuring user privacy and reducing risk of data misuse.
- **API Security:** Backend APIs are protected through controlled access mechanisms, and authentication can be implemented if required.
- **Access Control:** Only authorized components and services can interact with the system, preventing unauthorized usage.
- **Error Handling:** Proper error messages are used to avoid exposing internal system details, enhancing security.

7.2 Performance Aspects

- **Fast Response Time:** The system is optimized to generate responses within 2–3 seconds, ensuring real-time interaction.
- **Efficient Retrieval:** Vector databases enable quick similarity search, improving data retrieval speed.
- **Optimized Processing:** Embedding-based search reduces computational overhead compared to traditional methods.
- **Caching Mechanism:** Frequently asked queries and embeddings are cached to minimize repeated processing and improve speed.
- **Scalability Support:** The system can handle multiple users simultaneously without performance degradation.
- **Resource Optimization:** Efficient use of APIs and system resources reduces latency and operational cost.

8. References

- **RAG Architecture Concepts:** Reference to Retrieval-Augmented Generation techniques that combine information retrieval with language model generation to improve response accuracy.

- **Large Language Models (LLMs):** Documentation and usage guidelines of models such as OpenAI APIs for generating natural language responses.

- **Vector Databases:** Use of tools like FAISS and Pinecone for storing embeddings and performing similarity-based search efficiently.

- **REST API Design Principles:** Standard practices for designing scalable and maintainable APIs used in backend communication.

- **Python Documentation:** Reference to Python programming language and libraries used for backend development and AI integration.